Akademia Górniczo-Hutnicza w Krakowie			
Faculty: AGH	Academic Year 2025/2026	Year of study	Fields of science:
Student(s) name:	Simon GANIER-LOMBARD & Daniil STEPANOV		
Course:	**Real Time Operating Systems in Control Applications**		
Exercise nr:	**RTOS-05**		
Exercise subject:	**Basic interprocess synchronization: IPC, pulses, signals**		
Lecturer:	dr inż. Grzegorz Hayduk		
Date: 12.05.2026			Grade:

1 Exercise purpose

The goal of this exercise is for us to learn the basic ways to synchronize processes in QNX.
We extend the calculator from RTOS-03 with a new worker that computes the sine function every 100 ms.
This worker cannot be blocked on MsgReceive like the other ones, so the server uses pulses to ask it for the current value.

2 Assumptions / Theory

A pulse is a small message that the sender does not block on. Can carry 72 bits of payload (an 8-bit code and 64 bits of data)
The main functions are:
- MsgSendPulse sends a pulse on a connection without blocking.
- MsgReceivePulse receives a pulse on a channel.
- MsgReceive can also get a pulse;

The worker stays in its own loop and only reacts when a pulse arrives, so the sine value (for our case) keeps being computed without delay in the main loop.

3 Description of implementation

We reused the server and end-user client from RTOS-03 exeercice and added a new sine worker. The server was made generic so it handles both reply-driven workers (add, sub, mul, div) and pulse-driven workers (sin, in ou case it can be what we want) at the same time.
Architecture of the program
- ex5_server.c, the main server:

It keeps a list of workers and checks the ***chid*** field of the registration message: if chid is not zero the worker is pulse-driven, if ***chid*** is zero the worker is reply-driven.
For a pulse-driven worker the server saves the ***rcvid*** of the end-user client and calls ***MsgSendPulse*** to wake the worker, and replies to the end-user only when the worker sends back the answer with type ***a***.
- ex5_enduser_client.c:

It reads the input from the console, sends a message of type ***o*** to the server, and waits for the reply. The operator ***s*** asks for the sine value.
- ex5_sin_client.c the new worker (pulse-driven exemple):
It creates its own channel and a 100 ms cyclic timer that sends a ***TIMER_PULSE_CODE*** pulse to itself. On every tick the worker updates a counter and computes ***sin(counter)*** scaled by 1000. It registers with the server by sending its ***chid***, so the server can attach a connection and pulse it later. When the server sends ***MY_PULSE_CODE*** the worker sends back the current sine value with a message of type ***a***.
The sine worker uses ***MsgReceivePulse*** to wait on its own channel and a switch on pulse code to tell the two pulse types apart.
See the full commented code in the .zip file.

4 Conclusions

This exercise helped us understand the difference between blocking and non blocking synchronization. Via the implementation of reply-driven and a pulse-driven server/client.
The arithmetic workers use the reply-driven scheme and stay blocked on MsgSend until the server gives them a job. The sine worker cannot do that because it must keep computing every 100 ms, so the server pulses it instead.
The trickier part is to keep the server generic so that the same code routes both kinds of workers. We solved this by saving pulse_coid for pulse-driven workers and rcvid_wc for reply-driven workers in the same registration table.
The main advantage of pulses is that the worker is never blocked by the server and can do periodic work on its own clock.